

Assignment :- 3

Name : Kaushal Kumar Ray

Branch : B.Tech CSE with AI & ML

PRN : 240205242023

Semester : IV

Subject : TOC

- ① Explain CNF and GNF in brief. convert following grammar in CNF and GNF.

$S \rightarrow aAb / bB$

$A \rightarrow a / b$

$B \rightarrow b$

→ converting a CFG to CNF is an important step in many parsing algorithm like the CYK algorithm, and helps in understanding the structure of languages.

Conditions :

- A non-terminal generating a terminal (e.g. $X \rightarrow x$)
- A non-terminal generating two non-terminal (e.g. $X \rightarrow yz$)
- Starting symbol generating ϵ . (e.g. $S \rightarrow \epsilon$)
- ^{non}A^{terminal} generating two non-terminal (e.g. $A \rightarrow BC$)

Name: Kaushal Kumar Raj

PRN: 240205247023

CFG to CNF Given,

$S \rightarrow aAb / bB$

$A \rightarrow a / b$

$B \rightarrow b$

Here,

$S \rightarrow aAb / bB$

$a \rightarrow C$

$b \rightarrow D$

So,

$S \rightarrow CAD$

$S \rightarrow XD$

$X \rightarrow CA$

$\therefore S \rightarrow XD / DB$

$X \rightarrow CA$

$A \rightarrow a / b$

$B \rightarrow b$

$a \rightarrow C$

$b \rightarrow D$

Greibach Normal Form:

$\Rightarrow$ A CFG $\rightarrow$ CNF if all production rules satisfies one of the following conditions

• A Non-terminal generates a terminal (e.g. $X \rightarrow a$)

- A non-terminal generates a terminal followed by any number of non-terminal (e.g.: $X \rightarrow aABC$, $A \rightarrow bXY$, $B \rightarrow bA$) etc.

Now CF $\rightarrow$ GNF Given,

$$S \rightarrow aAb / bB$$

$$A \rightarrow a / b$$

$$B \rightarrow b$$

Solution,

$$S \rightarrow aAb / bB$$

$$B \rightarrow b$$

$$\therefore S \rightarrow aAB / bB$$

$$A \rightarrow a / b$$

$$B \rightarrow b$$

② Explain the formal definition of a Pushdown Automaton.

- $\rightarrow$ A pushdown Automaton is a finite automaton equipped with an extra memory structure called a stack. The stack works on the last-in-first-out (LIFO) principle and enables pushdown automata to recognize context-free language (CFLS). This article describes pushdown automata in detail on the next page:

Name: Kaushal Kumar Raj

PRN: 240205241023

It is a seven tuple machine

$$M = \{Q, \Sigma, \{q_0\}, \Gamma, \delta\}$$

where,

Q is the finite set of states

Σ is the set of input symbols

q_0 is the initial state

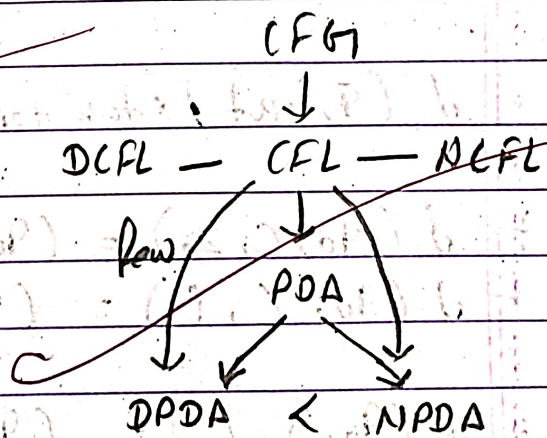
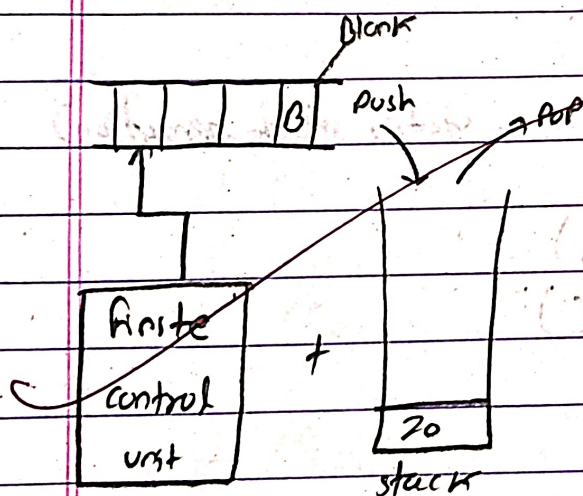
Γ is the set of final states

z_0 is the initial stack symbol present in the stack.

Γ is the set of pushdown (stack) symbol.

δ is the transition function that maps

$$\delta = (Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^*)$$



Turnstile Notation ($\vdash$)

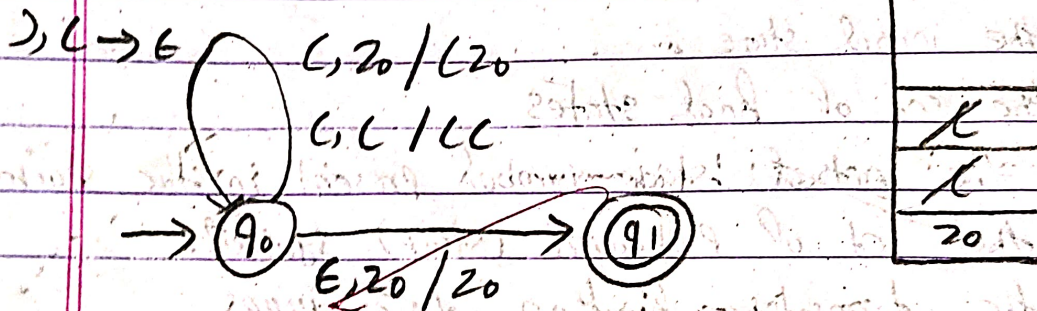
$\vdash$ sign is called a "turnstile notation" and represents one move and gives the instantaneous description.

③ Construct a PDA that accepts balanced parentheses.

Solution

Accepted string = $\{ (, (,)) \}$

minimum string = $\{ (,) \}$



Transition function

$\delta(q, \text{input} / \text{stack_top}) = (\text{new_state}, \text{stack_operation})$

$$\vdash \delta(q_0, C, Z_0) = (q_0, C Z_0)$$

$$\vdash \delta(q_0, C, C) = (q_0, CC)$$

$$\vdash \delta(q_0, \epsilon, Z_0) = (q_0, \epsilon)$$

$$\vdash \delta(q_0, \epsilon, Z_0) = (q_1, Z_0)$$

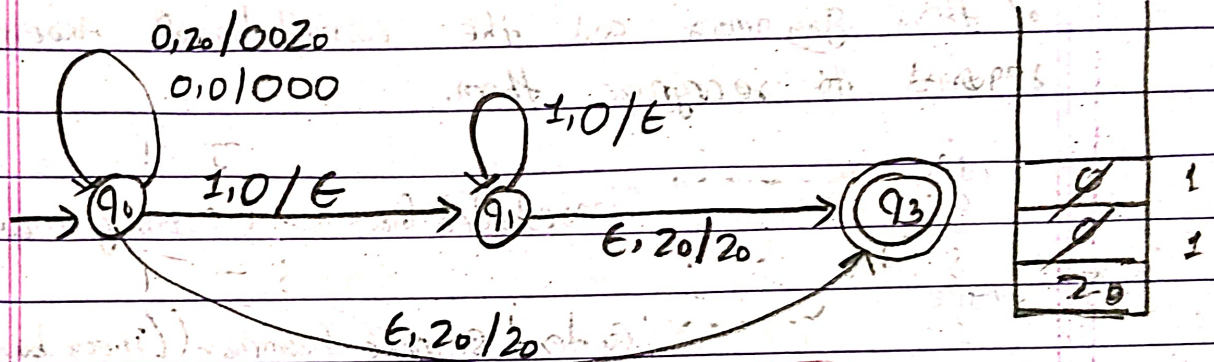
for 0 we push two 0's
for 1 we POP

Name: Kaushal Kumar Ray
PRN: 240205241023

Q) Design a PDA for the language $L = \{0^n 1^n \mid n \geq 0\}$

Solution,

$$\text{min string} = 0^1 1^1 \\ = 011$$



Transition Function:

$$\delta(q_0, 0, 20) = (q_0, 0020)$$

$$\delta(q_0, 0, 0) = (q_0, 000)$$

$$\delta(q_0, 1, 0) = (q_1, \epsilon)$$

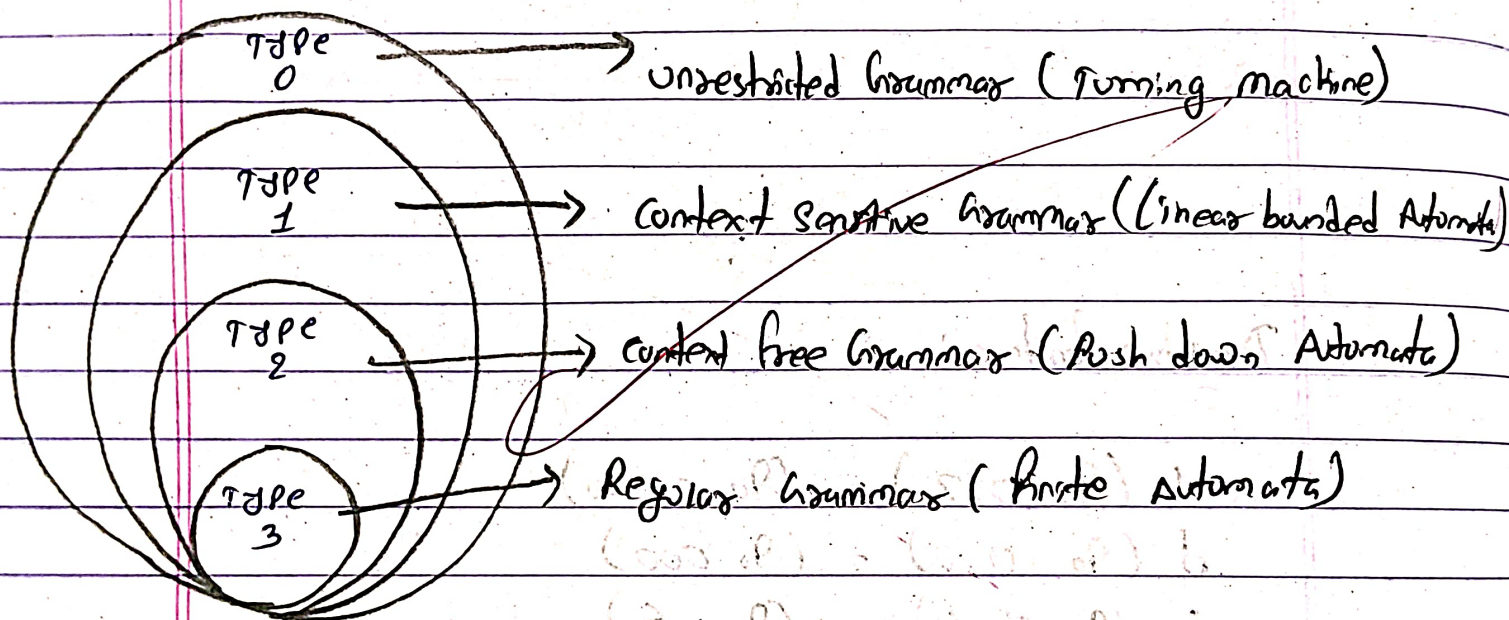
$$\delta(q_0, \epsilon, 20) = (q_3, 20)$$

$$\delta(q_1, 1, 0) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, 20) = (q_3, 20)$$

⑤ Explain chomsky hierarchy in brief. Construct a CFG for palindromes over $\{a, b\}$.

→ The chomsky hierarchy is a classification of formal language into four types based on the restrictions of their grammar and the computational power required to recognize them.



Explanation of types of grammar

① Type - 0 (URG)

$\alpha \rightarrow \beta$ where, $\alpha \in (V \cup T)^+ \neq \epsilon$ and $\beta \in (V \cup T)^*$

e.g.: $Sab \rightarrow ba$

$A \rightarrow \epsilon$

$asb \rightarrow sb$

$ab \rightarrow$

(i) Type-1 (CSG)

$$\alpha \rightarrow \beta, \quad \alpha \in (V+T)^* \quad V(V+T)^*$$

$$\beta \in (V+T)^*$$

$$\text{Here, } |\alpha| \leq |\beta|$$

Type 1 $\subset$ Type 0

$$\text{ex: } S \rightarrow AB, AB \rightarrow abc, B \rightarrow b, AS \rightarrow sb$$

(ii) Type-2 (CFG)

- $|\alpha| = 1$ Here,

$$\alpha \rightarrow \beta$$

$$\alpha \in V$$

$$\beta \in (V+T)^*$$

$$\text{where, } V = \{ \}$$

example:

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

(iii) Type 3 (R.G)

$$V \rightarrow VT / T$$

$$\text{eg: } A \rightarrow Aa / b$$

(Left Regular Grammar)

$$V \rightarrow TV / T$$

$$\text{eg: } A \rightarrow aA / b$$

(Right Regular Grammar)

Constructing CFG for Palindromes over $\{a, b\}$
Here,

accept strings: $a, b, aba, baab, abba,$
 $bababab$

$S \rightarrow aSa / bSb / a / b / \epsilon$

eg: $abababa$

$S \rightarrow a$

$a [babab] a$

$S \rightarrow b$

$b [aba] b$

$S \rightarrow aSa$

$a [b] a$

$absba$

$S \rightarrow bSb$

b

$abbbba$

$S \rightarrow b$

$S \rightarrow bSb$

$bbsbb$

$b b b b b$

To build a CFG for Palindromes, we use an "outside-in" recursive logic, if we have a valid Palindrome, we can create a new, longer Palindrome by attaching the same letter to both the beginning and the end